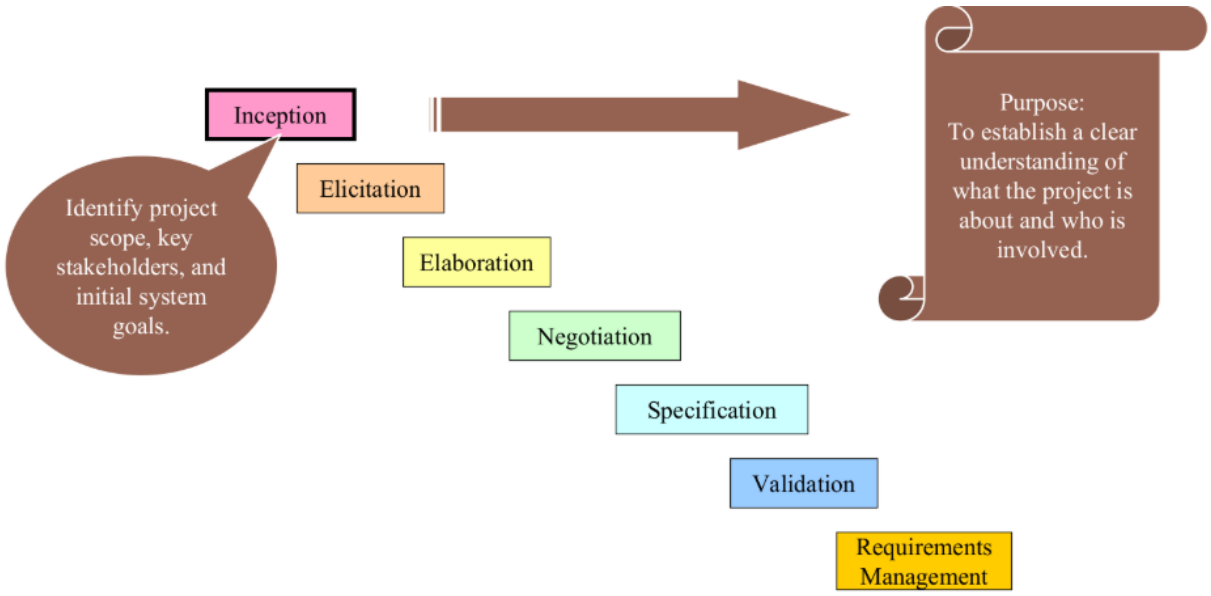
Requirements Engineering Tasks

Requirements Engineering is the process of establishing the services a system should provide and the constraints under which it must operate. It's a structured set of activities that transform an initial vague idea into a detailed, agreed-upon specification.



---

1. Inception

- **Goal:** To establish a basic understanding of the problem, the people who want a solution, the nature of the solution, and the scope of the project.
- **Key Question:** "What is the project really about?"
- **Activities:** Initial meetings with stakeholders, identifying high-level goals and constraints. It's the "starting the conversation" phase.

---

2. Elicitation

- **Goal:** To proactively discover the requirements from all relevant stakeholders.
- **Key Question:** "What do the users and other stakeholders need and want?"
- **Activities:** Conducting interviews, workshops, surveys, and observation. This is often the most difficult phase due to misunderstood, conflicting, or unstated needs.

Elicitation Activities: Elicitation may be accomplished through two activities:

A. Collaborative Requirements Gathering (CRG)

This is a meeting-centric approach designed to build consensus and identify problems through structured collaboration.

- **Core Idea:** Bring all key stakeholders (users, developers, customers, etc.) together in facilitated meetings to jointly define and review requirements.
- **Typical Process:**
    1. **Preparation:** A facilitator is chosen, and stakeholders are invited.
    2. **Meeting:** A structured discussion is guided by the facilitator.

3. **Idea Generation:** Participants brainstorm ideas, features, and constraints.
4. **Discussion & Negotiation:** Requirements are discussed, debated, and prioritized.
5. **Documentation:** Requirements are documented in real-time for immediate review and agreement.

- **Best For:** Projects where stakeholders have different perspectives that need to be aligned, and where direct communication is valuable.

B. Quality Function Deployment (QFD)

This is a more analytical, customer-driven approach that translates subjective customer needs into precise technical requirements.

- **Core Idea:** To ensure the final product reflects the customer's **voice** by linking their needs to every stage of development.

It identifies three types of requirements:

1. **Normal requirements:** These requirements are the objectives and goals stated for a product or system during meetings with the customer.
2. **Expected requirements:** These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them.
3. **Exciting requirements:** These requirements are for features that go beyond the customer's expectations and prove to be very satisfying when present.

Typical Process:

The basic idea of QFD is to construct relationship matrices between customer needs, technical requirements, priorities and (if needed) competitor assessment.

To achieve this the following process is prescribed:

- Identify stakeholder's attributes or requirements
- Identify technical features of the requirements
- Relate the requirements to the technical features
- Conduct an evaluation of competing products
- Evaluate technical features and specify a target value for each feature
- Prioritize technical features for development effort.

Best For: Product-driven environments (e.g., commercial software, automotive) where maximizing customer satisfaction and beating competitors are key goals.

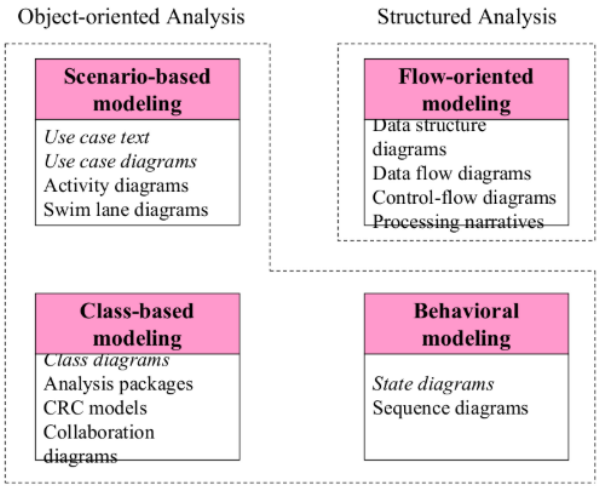
---

3. Elaboration

- **Goal:** To refine and model the elicited requirements to create a precise, consistent, and complete understanding.
- **Key Question:** "What exactly do these needs mean in detail?"
- **Activities:** Creating user stories, use cases, process models, data models, and prototypes to add detail and structure to the initial requirements.

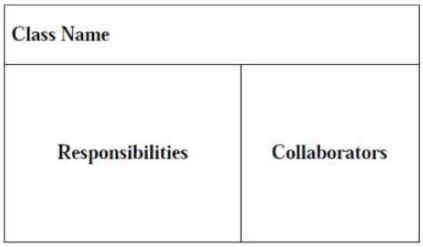
- Structured analysis
  - Considers **data** and the **processes** that transform the data as separate entities
  - Data is modeled in terms of only attributes and relationships (but no operations)
  - Processes are modeled to show the 1) input data, 2) the transformation that occurs on that data, and 3) the resulting output data

- Object-oriented analysis
  - Focuses on the definition of **classes** and the manner in which they **collaborate** with one another to fulfill customer requirements



CRC CARDS

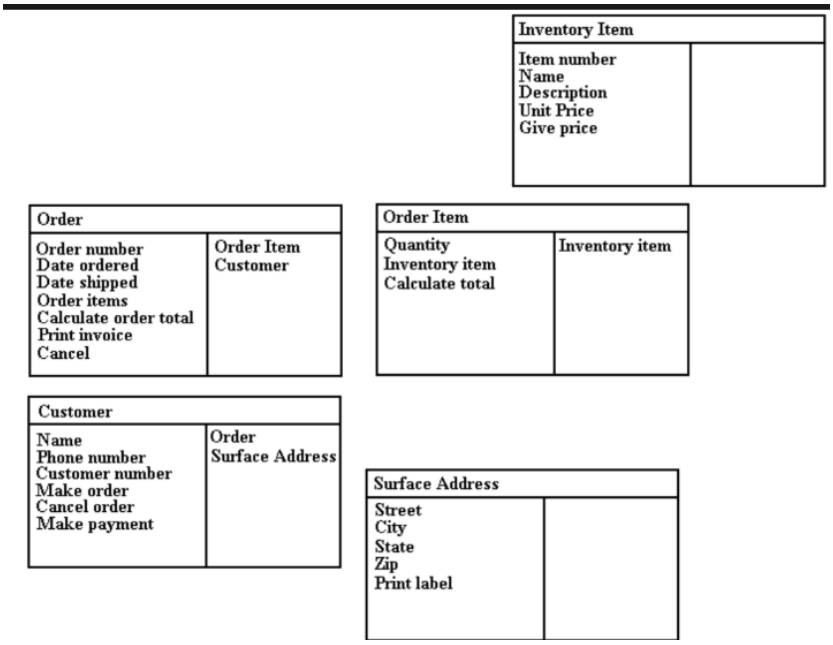
- A CRC card is divided into three sections (Beck & Cunningham, 1989; Ambler, 1995)



- A responsibility is anything that a class knows or does.
- Sometimes a class will have a responsibility to fulfill but will not have enough information to do it. When this happens, it has to collaborate with other classes to get the job done.

CRC MODEL

- A CRC model is a collection of CRC cards that represent whole or part of an application or problem domain.
- The most common use for CRC models is to gather and define the user requirements for an object-oriented application.
- Collaborating cards are placed close to each other



CREATING CRC MODEL

1. Put together the CRC modeling team.
2. Organize the modeling room.
3. Do some brainstorming.
4. Explain the CRC modeling technique.
5. Iteratively perform the steps of CRC modeling.
6. Perform use-case scenario testing.

CRC TEAM

- **Business Domain Experts (BDEs).**
- **Facilitator.**
- **Scribe(s).**
- **Observers.**

A) Room Preparation:

- Reserve a meeting room that has something write on.
- Bring CRC modeling supplies.
- Have a modeling table
- Have chairs and desks for the scribe(s).
- Have chairs for the BDEs.
- Have chairs for the observers

B) Brainstorming:

- Can we combine several jobs into one? Do we want to?
- How will people's jobs be affected? Are we empowering or disempowering them?
- What information will people need to do their jobs?
- Is work being performed where it makes the most sense?
- Are there any trivial tasks that we can automate?
- Are people performing only the complex tasks that the system can't handle?
- Will the system pay for itself?
- Does the system support teamwork, or does it hinder it?
- Do our users have the skills/education necessary to use this system? What training will they need?
- What are our organization's strategic goals and objectives? Does this system support them?
- How can we do this faster?
- How can we do this cheaper?

C) Explain the CRC Modeling Technique:

- Facilitators should describe the CRC modeling process. This usually takes between ten and fifteen minutes and will often include the creation of several example CRC cards.
- Lead BDEs through creation of example CRC cards

D) Perform the Iterative Steps for CRC Modeling:

- Find Classes:
    - Look for anything that interacts with the system, or is part of the system
    - Ask yourself “Is there a customer?”
    - Follow the money
    - Look for reports generated by the system
    - Look for any screens used in the system
    - Immediately prototype interface and report classes
    - Look for the three to five main classes right away
    - Create a new card for a class immediately

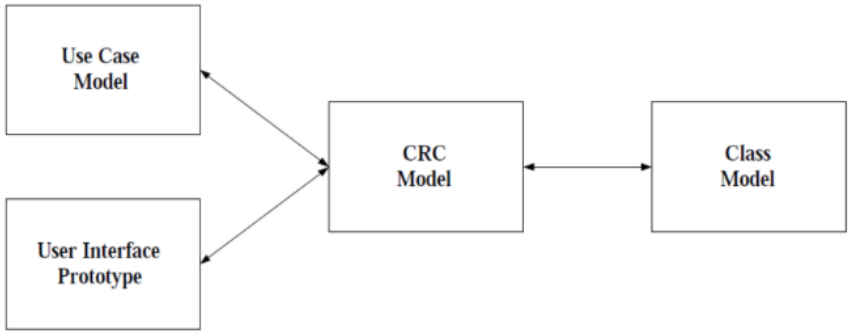
- Find responsibilities
    - Ask yourself what the class knows
    - Ask yourself what the class does
    - If you’ve identified a responsibility, ask yourself what class it "belongs" to
    - Sometimes we get responsibilities that we won’t implement, and that’s OK
    - Classes will collaborate to fulfill many of their responsibilities

- Define collaborators
    - Collaboration occurs when a class needs information that it doesn’t have
    - Collaboration occurs when a class needs to modify information that it doesn’t have
    - There will always be at least one initiator of any given collaboration
    - Sometimes the collaborator does the bulk of the work
    - New responsibilities may be created to fulfill the collaboration

E) Perform the Use Case Scenario Testing:

- Call out a new scenario (Select a new use case or new scenario)
- Determine which card should handle the responsibility
- Update the cards whenever necessary
- Describe the processing logic
- Collaborate if necessary
- Pass the ball back when done

HOW CRC MODELING FITS IN?



4. Negotiation

- **Goal:** To resolve conflicts between stakeholders regarding priorities, cost, schedule, and technical feasibility.
    - During negotiation, the software engineer reconciles the conflicts between what the customer wants and what can be achieved given limited business resources
- **Key Question:** "How do we balance different wants with what is possible?"

ART OF NEGOTIATION

- Recognize that it is **not competition**
- Map out a **strategy**
- Listen **actively**
- **Focus** on the other party's interests
- **D**on't let it get personal
- Be creative
- Be ready to commit

5. Specification

- **Goal:** To document the agreed-upon requirements in a formal, structured manner.
- **Activities:** Producing a Software Requirements Specification (SRS) document, which can be a text document, a set of detailed user stories with acceptance criteria, or a set of models.

SRC Template:

TABLE OF CONTENTS

6. Validation

- **Goal:** To ensure the requirements are defined correctly—that they are unambiguous, consistent, complete, and testable—and that they align with the project's goals.
- **Key Question:** "Did we define the *right* requirements correctly?"
- **Activities:** Conducting formal reviews (walkthroughs) and creating validation prototypes. Creating acceptance tests to verify the final product will meet the requirements.

7. Requirements Management

- **Goal:** To manage changes to the requirements throughout the project lifecycle in a controlled manner.

- **Key Question:** "How do we handle changing needs during the project?"
- **Activities:** Establishing a process for proposing, analyzing, approving, and implementing changes. Using traceability matrices to track the relationship between requirements and other work products (design, code, tests).

REQUIREMENTS MANAGEMENT TASK

- During requirements management, the project team performs a set of activities to identify, control, and track requirements and changes to the requirements at any time as the project proceeds

- Each requirement is assigned a unique identifier

- The requirements are then placed into one or more traceability tables

- These tables may be stored in a database that relate features, sources, dependencies, subsystems, and interfaces to the requirements

- A requirements traceability table is also placed at the end of the software requirements specification